

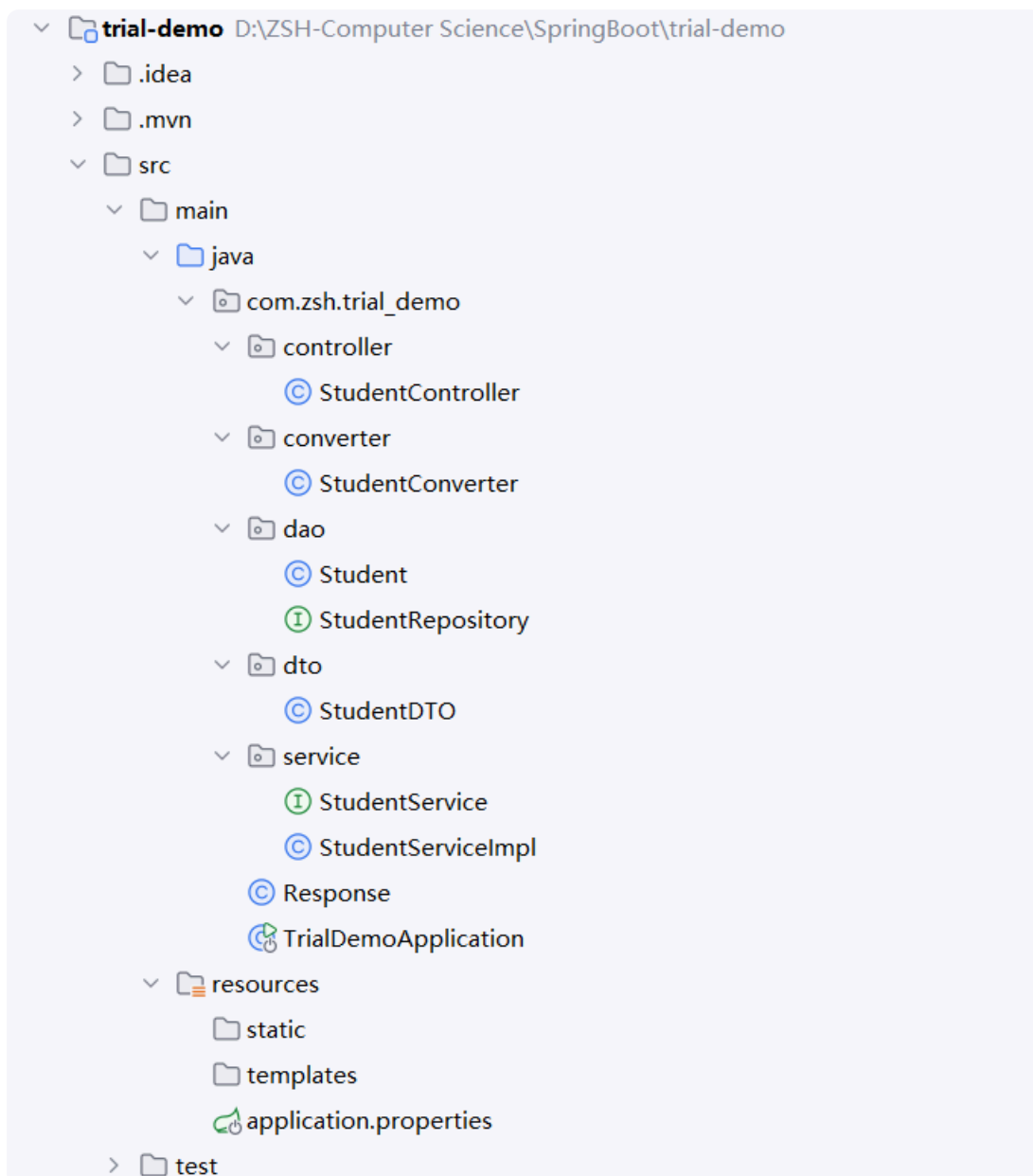
零基础SpringBoot小项目

该文章是本人观看B站up主“程序员风筑”于2024-04-23发布的“一小时带你从0到1实现一个SpringBoot项目开发”后，自行总结的笔记，视频链接如下：

https://www.bilibili.com/video/BV1gm411m7i6/?spm_id_from=333.337.search-card.all.click&vd_source=9837edfda9265147596b239f82083a53

项目结构分析

一、项目总览



- 1.首先新建一个 `SpringBoot-Maven` 工程，工程名称为 `trial-demo`
- 2.我们重点研究 `src-main` 下的结构
3. `main` 下面有两个包，一个是 `java` 包，用来存放我们的核心代码；另一个是 `resource` 包，用于存放我们的资源和配置文件

二、 `resource` 结构分析

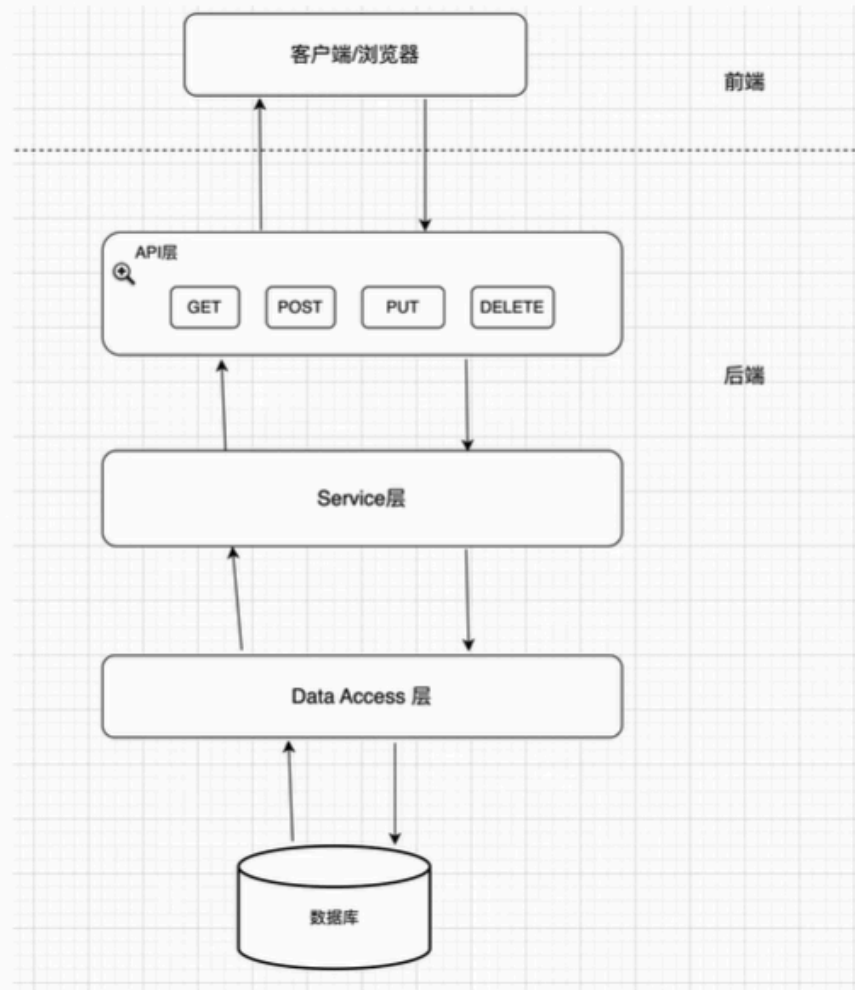
`application.properties` 里面配置了连接 `MySQL` 数据库的文件，如下：

```
1  spring.application.name=trial-demo
2
3  spring.datasource.url=jdbc:mysql://localhost:3306/test?
   characterEncoding=utf-8
4  spring.datasource.username=root
5  spring.datasource.password=123456
```

三、 `java-com.zsh.trial_demo` 结构分析

前后端交互时序图展示

项目结构



- `dao(mapper)`：持久层，用于将数据持久化，也就是向**数据库**中永久性地写入数据保存起来。
 - 持久层框架比如 `Mybatis` 可以简化持久层的开发，让我们专注于写 `SQL` 语句。
- `service`：逻辑层，用于实现业务数据的逻辑处理，可以向**持久层**要数据（无需关心数据库是什么类型），也可以向**接口层**提供经过逻辑处理后的数据。
- `controller(API)`：接口层，用于跟外部做交互，对外提供项目入口。

1. `dao` 包

- `Student` 类

```
1 package com.zsh.trial_demo.dao;
2
```

```
3 import jakarta.persistence.*;
4
5 @Entity//实体标注
6 @Table(name="student")//与数据库中相应的"student"表建立映射关系
7 public class Student {
8
9     @Id//主键标注
10    @GeneratedValue(strategy = GenerationType.IDENTITY)//自增
11    @Column(name="id")//与数据库中相应表的字段名建立映射关系
12    private long id;
13
14    @Column(name="name")//与数据库中相应表的字段名建立映射关系
15    private String name;
16
17    @Column(name="email")//与数据库中相应表的字段名建立映射关系
18    private String email;
19
20    @Column(name="age")//与数据库中相应表的字段名建立映射关系
21    private int age;
22
23    /*以下都是成员变量的get&set方法*/
24    public long getId() {
25        return id;
26    }
27    public void setId(long id) {
28        this.id = id;
29    }
30    public String getName() {
31        return name;
32    }
33    public void setName(String name) {
34        this.name = name;
35    }
36    public String getEmail() {
37        return email;
38    }
39    public void setEmail(String email) {
40        this.email = email;
41    }
42    public int getAge() {
43        return age;
44    }
45    public void setAge(int age) {
46        this.age = age;
47    }
```

- `StudentRepository` 接口

```
1 package com.zsh.trial_demo.dao;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5
6 @Repository//用于标记DAO组件，可以让Spring自动检测和管理这些DAO组件
7 public interface StudentRepository extends JpaRepository<Student,Long>
8 {
9     /*
10         JpaRepository<T,ID>是一个泛型接口，其中T代表泛型。
11         该接口拥有基本的CRUD（增删改查）功能。
12     */
13     //自定义了一个方法，通过邮箱查找拥有该邮箱的学生List集合
14     //只要自定义格式符合以下规范，那么java会为我们自动生成对应的方法，无需我们手
15     //写SQL语句
16     List<Student> findByEmail(String email);
17 }
```

2. `dto` 包

- `DT0(Data Transefer Object)`：数据传输对象
 - 作用：在不同层之间传输数据
 - `DT0` 类型的对象，不应该掺杂任何业务逻辑，只包含成员变量和相应的 `get&set` 方法
 - 通常在 `Service` 层使用，在 `Service` 层与 `Controller` 层之间传输数据
- `StudentDT0` 类

```
1 package com.zsh.trial_demo.dto;
2
3 public class StudentDT0 {
4
```

```

5      /*成员变量*/
6      private long id;
7      private String name;
8      private String email;
9
10     /*以下都是成员变量的get&set方法*/
11     public long getId() {
12         return id;
13     }
14     public void setId(long id) {
15         this.id = id;
16     }
17     public String getName() {
18         return name;
19     }
20     public void setName(String name) {
21         this.name = name;
22     }
23     public String getEmail() {
24         return email;
25     }
26     public void setEmail(String email) {
27         this.email = email;
28     }
29 }

```

3. converter 包

用于DAO和DTO之间的相互转换

```

1  package com.zsh.trial_demo.converter;
2
3  import com.zsh.trial_demo.dao.Student;
4  import com.zsh.trial_demo.dto.StudentDTO;
5
6  public class StudentConverter {
7
8      //将DAO从数据库中拿到的student对象，
9      //转换成我们想要返回给前端的studentDTO对象（序列化）
10     public static StudentDTO converterStudent(Student student){

```

```

11         StudentDTO studentDTO=new StudentDTO();
12         studentDTO.setId(student.getId());
13         studentDTO.setName(student.getName());
14         studentDTO.setEmail(student.getEmail());
15         return studentDTO;
16     }
17
18     //将前端拿到的studentDTO对象，
19     //转换成我们想要写入数据库的student对象（反序列化）
20     public static Student converterStudent(StudentDTO studentDTO){
21         Student student=new Student();
22         student.setId(studentDTO.getId());
23         student.setName(studentDTO.getName());
24         student.setEmail(studentDTO.getEmail());
25         return student;
26     }
27 }

```

4. Response 类

项目设计应遵守的规范，方便统一处理

```

1 package com.zsh.trial_demo;
2
3 public class Response <T>{//Response是一个泛型类
4
5     /*成员变量*/
6     private T data;//T代表泛型，所以data中存放的是泛型数据对象，比如Student对
象
7     private boolean success;//指示前端发送的请求是否成功
8     private String errorMsg;//记录错误信息
9
10    /*封装请求成功时返回的响应*/
11    public static <K> Response<K> newSuccess(K data){
12        //首先new一个Response对象出来
13        Response<K> response=new Response<>();
14

```

```
15         //再来设置这个Response对象的各种属性
16         response.setData(data);
17         response.setSuccess(true);
18
19         //最后返回这个Response对象
20         return response;
21     }
22
23     /*封装请求失败时返回的响应*/
24     public static Response<Void> newFail(String errorMsg){
25         //首先new一个Response对象出来
26         Response<Void> response=new Response<>();
27
28         //再来设置这个Response对象的各种属性
29         response.setSuccess(false);
30         response.setErrorMsg(errorMsg);
31
32         //最后返回这个Response对象
33         return response;
34     }
35
36     /*以下都是成员变量的get&set方法*/
37     public T getData() {
38         return data;
39     }
40     public void setData(T data) {
41         this.data = data;
42     }
43     public boolean isSuccess() {
44         return success;
45     }
46     public void setSuccess(boolean success) {
47         this.success = success;
48     }
49     public String getErrorMsg() {
50         return errorMsg;
51     }
52     public void setErrorMsg(String errorMsg) {
53         this.errorMsg = errorMsg;
54     }
55 }
```


5. service包

- StudentService 接口

```
1 package com.zsh.trial_demo.service;
2
3 import com.zsh.trial_demo.dao.Student;
4 import com.zsh.trial_demo.dto.StudentDTO;
5
6 public interface StudentService {
7     /*在接口中预先定义好准备实现的各种方法，如下：*/
8     //查
9     StudentDTO getStudentById(long id);
10    //增
11    Long addNewStudent(StudentDTO studentDTO);
12    //删
13    void deleteStudentById(long id);
14    //改
15    StudentDTO updateStudentById(long id, String name, String email);
16 }
```

- StudentServiceImpl 接口实现类

```
1 package com.zsh.trial_demo.service;
2
3 import com.zsh.trial_demo.converter.StudentConverter;
4 import com.zsh.trial_demo.dao.Student;
5 import com.zsh.trial_demo.dao.StudentRepository;
6 import com.zsh.trial_demo.dto.StudentDTO;
7 import jakarta.transaction.Transactional;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10 import org.springframework.util.CollectionUtils;
11 import org.springframework.util.StringUtils;
12
13 import java.beans.Transient;
14 import java.util.List;
15
16 @Service//逻辑层标注
```

```

17 public class StudentServiceImpl implements StudentService{
18
19     @Autowired//自动注入
20     private StudentRepository studentRepository;
21
22     /*以下是接口中预定义的方法的具体实现*/
23     //查
24     @Override
25     public StudentDTO getStudentById(long id) {
26         //先检查该id是否存在，若不存在则抛出RuntimeException异常
27         //findById是studentRepository继承自父类的方法，我们直接调用即可
28         Student student =
29 studentRepository.findById(id).orElseThrow(RuntimeException::new);
30
31         //返回将student序列化后的studentDTO对象
32         return StudentConverter.converterStudent(student);
33     }
34
35     //增
36     @Override
37     public Long addNewStudent(StudentDTO studentDTO) {
38         //先检查新增学生的email是否已被他人占用
39         //此处调用的findByEmail方法是一个自定义方法，但由于我们自定义格式符合规范，故无需手写SQL语句，java会帮我们自动生成该方法
40         List<Student>
41 studentList=studentRepository.findByEmail(studentDTO.getEmail());
42         if(!CollectionUtils.isEmpty(studentList)){
43             throw new
44 IllegalStateException("email:"+studentDTO.getEmail()+" has been
45 taken.");
46         }
47
48         //将新增的studentDTO对象序列化为student对象，再保存进数据库
49         //save是studentRepository继承自父类的方法，我们直接调用即可
50         Student
51 student=studentRepository.save(StudentConverter.converterStudent(student
52 tDTO));
53
54         //返回序列化后的student对象的id
55         return student.getId();
56     }
57
58     //删
59     @Override
60     public void deleteStudentById(long id) {

```

```

55         //先检查该id是否存在，若不存在则抛出异常
56         studentRepository.findById(id).orElseThrow(() ->
57             new IllegalArgumentException("id:"+id+" doesn't
exist!"));
58
59         //deleteById是studentRepository继承自父类的方法，我们直接调用即可
60         studentRepository.deleteById(id);
61     }
62
63     //改
64     @Transactional//若更新失败，则进行事务回滚
65     @Override
66     public StudentDTO updateStudentById(long id, String name, String
email) {
67         //先检查该id是否存在，若不存在则抛出异常
68         Student
studentInDB=studentRepository.findById(id).orElseThrow(() ->
69             new IllegalArgumentException("id:"+id+" doesn't
exist!"));
70
71         //如果传进来的name不为空且与数据库中的name不相同，我们才更新数据库中的
name
72         if(StringUtils.hasLength(name) &&
!studentInDB.getName().equals(name)){
73             studentInDB.setName(name);
74         }
75
76         //如果传进来的email不为空且与数据库中的email不相同，我们才更新数据库中的
email
77         if(StringUtils.hasLength(email) &&
!studentInDB.getEmail().equals(email)){
78             studentInDB.setEmail(email);
79         }
80         //将修改后的student对象保存进数据库
81         //save是studentRepository继承自父类的方法，我们直接调用即可
82         Student student=studentRepository.save(studentInDB);
83
84         //返回将student反序列化后的studentDTO对象
85         return StudentConverter.converterStudent(student);
86     }
87 }

```

6. controller 包

易错!!! `public Response<Long> addNewStudent(@RequestBody StudentDTO studentDTO)`

注意此处必须要有 `RequestBody` 标注, 否则前后端无法交互, 导致 `add` 接口失效

- `StudentController` 类

```
1 package com.zsh.trial_demo.controller;
2
3 import com.zsh.trial_demo.Response;
4 import com.zsh.trial_demo.dao.Student;
5 import com.zsh.trial_demo.dto.StudentDTO;
6 import com.zsh.trial_demo.service.StudentService;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.web.bind.annotation.*;
9
10 @RestController//接口层标注
11 public class StudentController {
12
13     @Autowired//自动注入
14     private StudentService studentService;
15
16     /*以下是增删改查4个接口*/
17     //查
18     @GetMapping("/student/{id}")//GET方法标注
19     public Response<StudentDTO> getStudentById(@PathVariable long id)
20     { //路径变量标注, 指代{id}
21         //调用Service层中的相关方法处理请求, 并将响应结果封装成一个响应对象返回
22         return Response.newSuccess(studentService.getStudentById(id));
23     }
24
25     //增
26     @PostMapping("/student")//POST方法标注
27     public Response<Long> addNewStudent(@RequestBody StudentDTO
28     studentDTO){ //RequestBody标注
29         //调用Service层中的相关方法处理请求, 并将响应结果封装成一个响应对象返回
30         return
31         Response.newSuccess(studentService.addNewStudent(studentDTO));
32     }
33 }
```

```

30
31     //删
32     @DeleteMapping("/student/{id}")//DELETE方法标注
33     public void deleteStudentById(@PathVariable long id){//路径变量标注,
指代{id}
34         //调用Service层中的相关方法处理请求,并将响应结果封装成一个响应对象返回
35         studentService.deleteStudentById(id);
36     }
37
38     //改
39     @PutMapping("/student/{id}")//PUT方法标注
40     public Response<StudentDTO> updateStudentById
41         (@PathVariable long id,//路径变量标注, 指代{id}
42         @RequestParam(required = false)String name,//请求参数name标
注
43         @RequestParam(required = false)String email){//请求参数
email标注
44         //调用Service层中的相关方法处理请求,并将响应结果封装成一个响应对象返回
45         return
Response.newSuccess(studentService.updateStudentById(id,name,email));
46     }
47 }

```

7. TrialDemoApplication 类

项目启动入口

```
1 package com.zsh.trial_demo;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class TrialDemoApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(TrialDemoApplication.class, args);
10    }
11 }
```